SMART FARMING INTELLIGENCE PLATFORM

Team Members:

Madhavan S (127006021)

Mohamed Ibrahim K (12700415)

Mythili S (127006039)

Guided by:

Dr.Arun K

Asst.Professor-III

arun@eee.sastra.edu

OBJECTIVE

*To monitor real-time soil and environmental parameters using IoT sensors.
*To design an offline-first smart farming system suitable for rural areas.
*To enable long-range communication using LoRa technology.
*To provide farmer alerts through GSM without internet dependency.
*To ensure low-cost, scalable, and reliable agricultural monitoring.

PROBLEM STATEMENT:

- Agriculture in rural India still depends heavily on experience-based decision making due to the lack of real-time field data and reliable connectivity. Farmers face challenges such as improper irrigation, soil degradation, unpredictable weather, and delayed response to critical field conditions.
- Most existing smart farming solutions depend on continuous internet connectivity or cloud-based processing, making them unreliable in low-connectivity or offline rural regions.
- Hence, there is a need for a low-cost, offline-first intelligent farming system that can monitor soil and environmental parameters locally and provide timely alerts to farmers without relying on the internet.

MOTIVATION

Indian agriculture faces major challenges due to climate uncertainty and lack of data-driven farming practices, especially in rural regions with limited Internet access. Farmers often rely on manual observation, leading to inefficient irrigation and reduced crop productivity.

Most existing smart agriculture solutions are cloud-dependent and expensive, making them unsuitable for small-scale farmers. Therefore, a practical, offline-capable smart farming system is essential to support real-world agricultural conditions in India.

LITERATURE SURVEY

((IoT-Based Smart Agriculture Systems))

These systems monitor soil and weather parameters using sensors and cloud platforms. However, continuous internet dependency limits their effectiveness in rural regions.

((Machine Learning-Based Agricultural Systems))

ML techniques assist in crop prediction and decision-making but often require cloud computation and high processing resources.

((LoRa-Based Agricultural Monitoring))

LoRa communication enables long-range, low-power data transmission but is commonly used without local intelligence or offline processing.

((Research Gap))

There is a lack of systems that combine offline processing, long-range communication, and farmer alert mechanisms in a single unified platform.

PROJECT TIMELINE

#STEP 1

Analysis & System Design

Problem Identification

System architecture design

Component selection

STEP 2

Development & Implementation

Hardware integration

Firmware development

communication setup

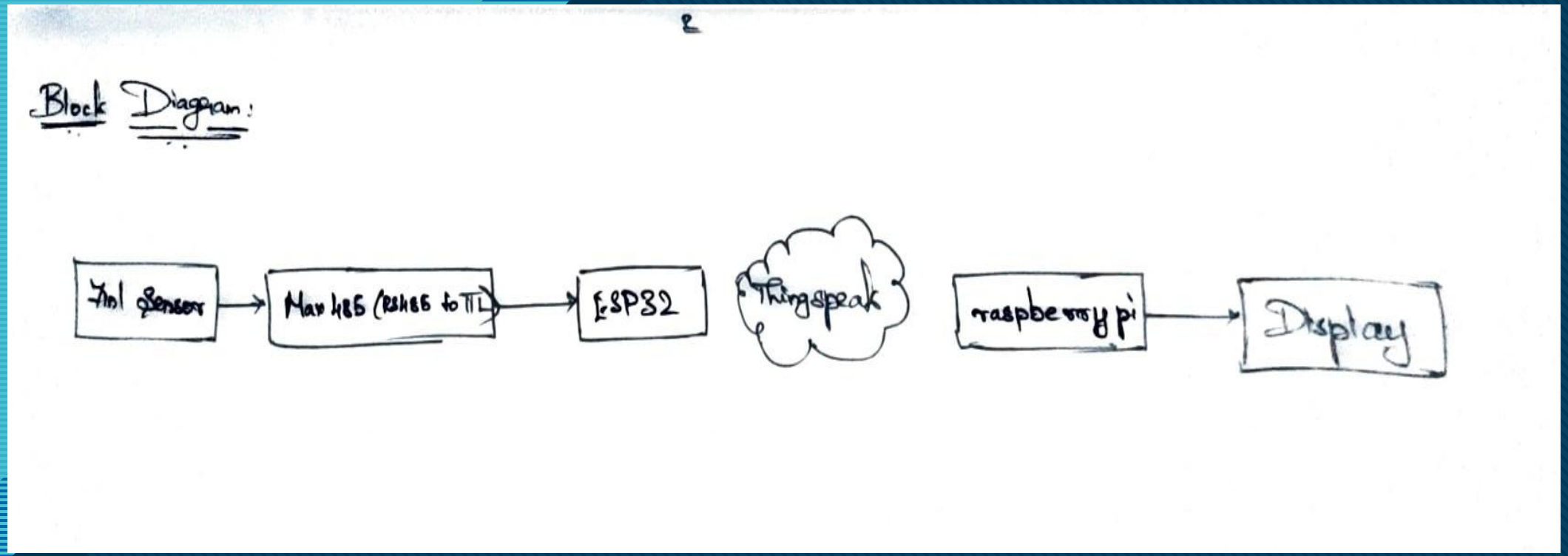
#STEP 3

Integration, Testing & Documentation

System testing and validation

Documentation and review preparation

BLOCK DIAGRAM



CODE :

```
import pandas as pd
import numpy as np
import random
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.graph_objects as go
import plotly.express as px
from plotly.subplots import make_subplots

colorarr = []

cropdf = pd.read_csv("../input/crop-recommendation-dataset/Crop_recommendation.csv")
cropdf.head()

cropdf.shape

cropdf.columns

cropdf.isnull().any()

print("Number of various crops: ", len(cropdf['label'].unique()))
print("List of crops: ", cropdf['label'].unique())

cropdf['label'].value_counts()

crop_summary = pd.pivot_table(cropdf,index=['label'],aggfunc='mean')
crop_summary.head()

crop_summary_N = crop_summary.sort_values(by='N', ascending=False)
fig = make_subplots(rows=1, cols=2)

top = {
   'y' : crop_summary_N['N'][0:10].sort_values().index,
   'x' : crop_summary_N['N'][0:10].sort_values()
}

last = {
   'y' : crop_summary_N['N'][-10:].index,
   'x' : crop_summary_N['N'][-10:]
}

fig.add_trace(
   go.Bar(top,
        name="Most nitrogen required",
```

```
fig.add_trace(
   go.Bar(last,
        name="Least phosphorus required",
        marker_color=random.choice(colorarr),
        orientation='h',
        text=last['x']),
   row=1, col=2
)

fig.update_traces(texttemplate='%{text}', textposition='inside')
fig.update_layout(title_text="Phosphorus
(P)",plot_bgcolor='white',font_size=12,font_color='black',height=500)
fig.update_xaxes(showgrid=False)
fig.update_yaxes(showgrid=False)
fig.show()

crop_summary_K = crop_summary.sort_values(by='K', ascending=False)
fig = make_subplots(rows=1, cols=2)

top = {
   'y' : crop_summary_K['K'][0:10].sort_values().index,
   'x' : crop_summary_K['K'][0:10].sort_values()
}

last = {
   'y' : crop_summary_K['K'][-10:].index,
   'x' : crop_summary_K['K'][-10:]
}

fig.add_trace(
   go.Bar(top,
        name="Most potassium required",
        marker_color=random.choice(colorarr),
        orientation='h',
        text=top['x']),
   row=1, col=1
)

fig.add_trace(
   go.Bar(last,
        name="Least potassium required",
        marker_color=random.choice(colorarr),
        orientation='h',
        text=last['x']),
```

```
        marker_color=random.choice(colorarr),
        orientation='h',
        text=top['x']),
   row=1, col=1
)

fig.add_trace(
   go.Bar(last,
        name="Least nitrogen required",
        marker_color=random.choice(colorarr),
        orientation='h',
        text=last['x']),
   row=1, col=2
)

fig.update_traces(texttemplate='%{text}', textposition='inside')
fig.update_layout(title_text="Nitrogen
(N)",plot_bgcolor='white',font_size=12,font_color='black',height=500)
fig.update_xaxes(showgrid=False)
fig.update_yaxes(showgrid=False)
fig.show()

crop_summary_P = crop_summary.sort_values(by='P', ascending=False)
fig = make_subplots(rows=1, cols=2)

top = {
   'y' : crop_summary_P['P'][0:10].sort_values().index,
   'x' : crop_summary_P['P'][0:10].sort_values()
}

last = {
   'y' : crop_summary_P['P'][-10:].index,
   'x' : crop_summary_P['P'][-10:]
}

fig.add_trace(
   go.Bar(top,
        name="Most phosphorus required",
        marker_color=random.choice(colorarr),
        orientation='h',
        text=top['x']),
   row=1, col=1
)
```

CODE :

```
lentil_npk = crop_summary[crop_summary.index=='lentil']
values = [lentil_npk['N'][0], lentil_npk['P'][0], lentil_npk['K'][0]]
fig.add_trace(go.Pie(labels=labels, values=values,name="Lentil"),1, 5)

fig.update_traces(hole=.4, hoverinfo="label+percent+name")
fig.update_layout(title_text="NPK ratio for rice, cotton, jute, maize, lentil")
fig.show()

crop_scatter = cropdf[(cropdf['label']=='rice') | (cropdf['label']=='jute') |
(cropdf['label']=='cotton') | (cropdf['label']=='maize') | (cropdf['label']=='lentil')]

fig = px.scatter(crop_scatter, x="temperature", y="humidity", color="label", symbol="label")
fig.update_layout(plot_bgcolor='white')
fig.update_xaxes(showgrid=False)
fig.update_yaxes(showgrid=False)
fig.show()

fig = px.bar(crop_summary, x=crop_summary.index, y=["rainfall", "temperature",
"humidity"])
fig.update_layout(title_text="Comparision between rainfall, temerature and
humidity",plot_bgcolor='white',height=500)
fig.update_xaxes(showgrid=False)
fig.update_yaxes(showgrid=False)
fig.show()

fig, ax = plt.subplots(1, 1, figsize=(15, 9))
sns.heatmap(cropdf.corr(), annot=True,cmap='Wistia')
ax.set(xlabel='features')
ax.set(ylabel='features')
plt.title('Correlation between different features')
plt.show()

X = cropdf.drop('label', axis=1)
y = cropdf['label']

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.3, shuffle = True,
random_state = 0)

import lightgbm as lgb

model = lgb.LGBMClassifier()
```

```
  row=1, col=2
)

fig.update_traces(texttemplate='%{text}', textposition='inside')
fig.update_layout(title_text="Potassium
(K)",plot_bgcolor='white',font_size=12,font_color='black',height=500)
fig.update_xaxes(showgrid=False)
fig.update_yaxes(showgrid=False)
fig.show()

fig = go.Figure()

fig.add_trace(go.Bar(x=crop_summary.index,y=crop_summary['N'],name='Nitrogen',marker
_color='indianred'))
fig.add_trace(go.Bar(x=crop_summary.index,y=crop_summary['P'],name='Phosphorous',mar
ker_color='lightsalmon'))
fig.add_trace(go.Bar(x=crop_summary.index,y=crop_summary['K'],name='Potash',marker_c
olor='crimson'))

fig.update_layout(title="N, P, K values comparision between
crops",plot_bgcolor='white',barmode='group',xaxis_tickangle=-45)
fig.show()

labels = ['Nitrogen(N)','Phosphorous(P)','Potash(K)']

fig = make_subplots(rows=1, cols=5,
specs=[[{'type':'domain'},{'type':'domain'},{'type':'domain'},{'type':'domain'},{'type':'domain'
}]])

rice_npk = crop_summary[crop_summary.index=='rice']
values = [rice_npk['N'][0], rice_npk['P'][0], rice_npk['K'][0]]
fig.add_trace(go.Pie(labels=labels, values=values,name="Rice"),1, 1)

cotton_npk = crop_summary[crop_summary.index=='cotton']
values = [cotton_npk['N'][0], cotton_npk['P'][0], cotton_npk['K'][0]]
fig.add_trace(go.Pie(labels=labels, values=values,name="Cotton"),1, 2)

jute_npk = crop_summary[crop_summary.index=='jute']
values = [jute_npk['N'][0], jute_npk['P'][0], jute_npk['K'][0]]
fig.add_trace(go.Pie(labels=labels, values=values,name="Jute"),1, 3)

maize_npk = crop_summary[crop_summary.index=='maize']
values = [maize_npk['N'][0], maize_npk['P'][0], maize_npk['K'][0]]
fig.add_trace(go.Pie(labels=labels, values=values,name="Maize"),1, 4)
```

```
model.fit(X_train, y_train)

y_pred=model.predict(X_test)

from sklearn.metrics import accuracy_score

accuracy=accuracy_score(y_pred, y_test)
print('LightGBM Model accuracy score: {0:0.4f}'.format(accuracy_score(y_test, y_pred)))

y_pred_train = model.predict(X_train)
print('Training-set accuracy score: {0:0.4f}'. format(accuracy_score(y_train, y_pred_train)))

print('Training set score: {:.4f}'.format(model.score(X_train, y_train)))
print('Test set score: {:.4f}'.format(model.score(X_test, y_test)))

from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(15,15))
sns.heatmap(cm, annot=True, fmt=".0f", linewidths=.5, square = True, cmap = 'Blues')
plt.ylabel('Actual label')
plt.xlabel('Predicted label')
plt.title('Confusion Matrix')
plt.show()

from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))

newdata=model.predict([[90, 42, 43, 20.879744, 75, 5.5,220]])
newdata
```

Crop Recommendation System (ML-Based):

Objective:
Recommend the best crop using soil nutrients & weather data

Inputs:
- N, P, K
- Temperature
- Humidity
- pH
- Rainfall

Method:
Data Analysis & Visualization
Train/Test Split (70/30)
Model: LightGBM Classifier

Prediction Example :
Input → (90, 42, 43, 20.8, 75, 5.5, 220)
Output → Rice

Applications :
Smart Agriculture • Precision Farming • Crop Advisory Systems

OVERALL PIPELINE :

```
Dataset
   ↓
Data Analysis
   ↓
Visualization
   ↓
Feature Selection
   ↓
Train/Test Split
   ↓
LightGBM Model Training
   ↓
Model Evaluation
   ↓
Crop Prediction
```

Thank You